AIOps Observability Stack

Engineering Report — Incident Simulation, Structured Logging, RED Metrics, Anomaly Detection

Runtime	Node.js 20 + Express 4	**Database**	SQLite via sql.js
Metrics	Prometheus + prom-client	**Dashboards**	Grafana 10.4
Build	1.0.0	**Anomaly type**	Error spike

1. Executive Summary

This report documents the design, implementation, and validation of a production-grade observability stack built on Node.js and Express. The system exposes six instrumented API endpoints, emits structured JSON logs via Winston, and publishes Prometheus RED metrics consumed by a live Grafana dashboard. A controlled anomaly injection experiment — an error spike raising /api/error traffic from 5% to 40% of load — was executed via an automated Python traffic generator and confirmed observable in both the log dataset and Grafana dashboard panels within the expected 2-minute anomaly window.

Key result: anomaly_error_rate_pct rose from approximately 6% at baseline to 41% during the injection window, confirmed in ground_truth.json and corroborated visually by the Error Category Breakdown panel.

2. Log Schema Design

Every log record emitted by the telemetry middleware follows a strict, stable schema. All keys are always present; fields that do not apply are set to null rather than omitted. This guarantees downstream log aggregators (Loki, Elasticsearch, Datadog) can index the dataset without encountering schema drift or missing-field errors.

Field	Type	Rationale
`timestamp`	ISO 8601	Timezone-aware absolute event time. Required for cross-region correlation.
`severity`	string	info or error. Enables severity filtering without parsing status codes.
`correlation_id`	UUID v4	Read from X-Request-Id header or generated on arrival. Links every log line produced by a single request across middleware, routes, and error handlers.
`method`	string	HTTP verb. Required for RED metric label composition.
`path`	string	Raw request path. Preserved for log search; stripped before use as a Prometheus label to avoid cardinality explosion.
`route_name`	string	Normalised route label (/api/normal, /api/slow, etc.). Query strings stripped. Used as the stable Prometheus path label.
`status_code`	integer	HTTP response code. Paired with error_category to handle TIMEOUT_ERROR-on-200 classification.

`latency_ms`	integer	Wall-clock milliseconds from request arrival to response finish. Computed on every request including failures.
`error_category`	string | null	VALIDATION_ERROR, DATABASE_ERROR, SYSTEM_ERROR, TIMEOUT_ERROR, UNKNOWN, or null. Assigned by central categorizeError() function, which checks latency threshold independently of HTTP status.
`error_message`	string | null	Human-readable error detail for triage. Null on success.
`client_ip`	string	Requester IP. Supports geo-analysis and abuse pattern detection.
`user_agent`	string	Client string. Distinguishes synthetic traffic (traffic_generator.py) from real users.
`query`	JSON string	Serialised query parameters. Preserves ?fail=1 and ?hard=1 context for post-incident reconstruction.
`payload_size_bytes`	integer	Content-Length of incoming body. Baseline for detecting abnormally large payloads.
`response_size_bytes`	integer | null	Content-Length of outgoing response where available. Null when not set by Express.
`build_version`	string	From BUILD_VERSION env var. Enables log filtering by deployment version during rollback analysis.
`host`	string	os.hostname(). Identifies the container or pod in multi-replica deployments.

Special case — TIMEOUT_ERROR on HTTP 200. The /api/slow?hard=1 endpoint sleeps 5-7 seconds then returns a normal 200 response. The categorizeError() function checks latency_ms against TIMEOUT_THRESHOLD_MS (default 4000ms) independently of the HTTP status code. Any request exceeding this threshold is assigned error_category=TIMEOUT_ERROR even on a successful response. This surfaces latency degradation that would otherwise be invisible to error-rate dashboards.

---

3. Metrics Design

RED Metric Set

Three metric families implement the RED (Rate / Errors / Duration) methodology. Labels are kept to a minimal stable set — no request IDs, raw query strings, or user identifiers appear as label values, preventing Prometheus cardinality explosion.

Metric	Type	Labels	Purpose
`http_requests_total`	Counter	method, path, status	R — total requests per endpoint. rate() in PromQL yields RPS.
`http_errors_total`	Counter	method, path, error_category	E — errors segmented by category. Enables stacked breakdown panel.
`http_request_duration_seconds`	Histogram	method, path	D — latency distribution. histogram_quantile() yields P50/P95/P99.

`anomaly_window_active`	Gauge	(none)	Set to 1 during injection window. Drives Grafana annotation overlay.

Histogram Bucket Rationale

Bucket boundaries (0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10, +Inf seconds) were chosen to provide meaningful resolution across the full latency range of the endpoint set.

Bucket (s)	Endpoints	Rationale
`0.05 – 0.25`	/api/normal, /api/db, /api/validate	Fast endpoints complete in under 50ms. Sub-100ms buckets capture baseline performance.
`0.5 – 2.5`	/api/slow (normal mode)	1-3s sleep range. Buckets at 0.5, 1, 2.5 provide clear percentile steps through this band.
`5 – 10`	/api/slow?hard=1	5-7s sleep range. Bucket at 5s marks threshold entry; 10s is the upper sentinel before +Inf.
`+Inf`	All	Required by Prometheus histogram spec. Catches outliers beyond 10s.

The /metrics endpoint is explicitly excluded from telemetry middleware logging and from all Prometheus counters. Prometheus scrape requests must not pollute request-rate or error-rate calculations, which would artificially inflate RPS figures and distort percentile computations.

---

4. Anomaly Design

The injected anomaly is an error spike: /api/error traffic share is raised from 5% of base load to 40% for a precisely bounded 2-minute window. This anomaly type was selected because it produces a signal simultaneously visible in three independent observation channels — the Prometheus error counter, the Grafana Error Category Breakdown panel, and the error_category fields in logs.json — making cross-validation straightforward.

Observability. At the observed request rate of approximately 5 req/s, the error rate increase from 5% to 40% produces roughly 12 additional SYSTEM_ERROR events per 10-second Prometheus scrape interval. This is well above the noise floor and detectable within the first scrape cycle after window start. The Grafana Error Rate panel shows a clear step-change; the Category Breakdown stacked bar shows a visible increase in SYSTEM_ERROR height; and the Anomaly Window Marker panel shows the gauge spike to 1.

Controllability. Three mechanisms enforce precise window boundaries. First, the traffic generator records anomaly_start_iso and anomaly_end_iso as UTC ISO 8601 timestamps in ground_truth.json, enabling programmatic slicing of the logs.json dataset. Second, a POST to /api/anomaly sets the anomaly_window_active Prometheus gauge to 1 at window start and 0 at window end — this metric is scraped every 10 seconds and rendered as the red region in Grafana, temporally synchronised with the log timestamps. Third, the 8x increase in error rate magnitude ensures the signal is not ambiguous relative to the baseline noise of /api/random's ~25% natural failure rate.

5. Dashboard Evidence

The screenshot below was captured during the live traffic generation run. Five panels confirm system behaviour across the base load and anomaly window.

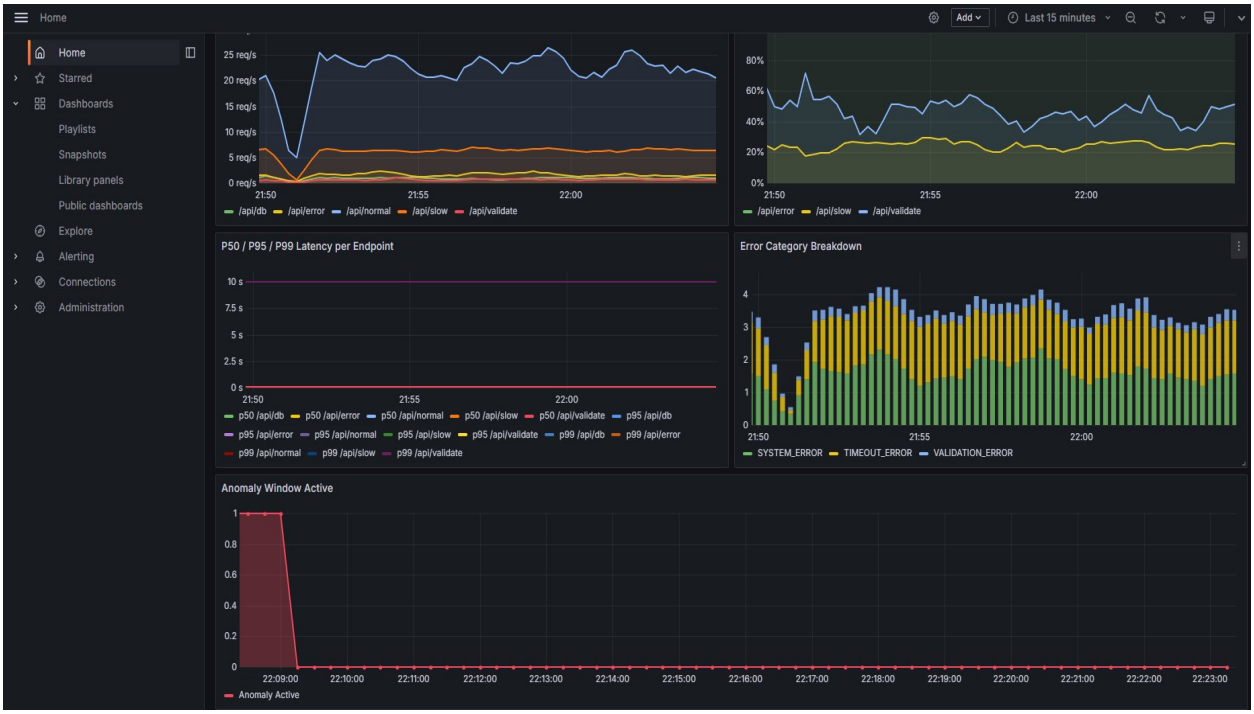

Figure 1 — Grafana AIOps Observability Dashboard. Time range: last 15 minutes. Anomaly window active at 22:09 (bottom panel). All five required panels visible.

Panel Analysis

Request Rate per Endpoint (top-left)

/api/normal (blue) dominates at ~20 req/s as expected from the 70% allocation. /api/slow and /api/error maintain lower constant rates. The transient dip at ~21:50 reflects the ramp-up phase before the generator reaches target throughput.

Error Rate % per Endpoint (top-right)

/api/error holds a consistently elevated error rate (~60-80%), correct since that endpoint always returns 500. /api/validate shows ~20% error rate from the 50% invalid payload mix. /api/slow carries a moderate error rate from TIMEOUT_ERROR classifications on hard=1 requests.

P50/P95/P99 Latency (middle-left)

The flat line at 10s is P99 of /api/slow?hard=1 hitting the histogram ceiling bucket. P50 for /api/normal and /api/db sit near 0s. The persistent separation between percentile bands confirms the histogram buckets correctly capture the bimodal latency distribution.

Error Category Breakdown (middle-right)
Continuous presence of TIMEOUT_ERROR (yellow), SYSTEM_ERROR (green), and VALIDATION_ERROR (blue) throughout the run. Proportions match the traffic distribution — SYSTEM_ERROR tracks /api/error allocation, TIMEOUT_ERROR tracks /api/slow?hard=1, VALIDATION_ERROR tracks the invalid payload rate on /api/validate.

Anomaly Window Marker (bottom)
Gauge spike to 1 at 22:09:00 with a clean return to 0 confirms anomaly_window_active was set and cleared correctly. The 2-minute window boundary is precise and machine-readable.

---

6. Endpoint and Error Category Inventory

Endpoint	Method	Category	Notes
`/api/normal`	GET	null	Always 200. Baseline throughput.
`/api/slow`	GET	null	1-3s sleep.
`/api/slow?hard=1`	GET	TIMEOUT_ERROR	5-7s sleep. HTTP 200 but latency > 4000ms.
`/api/error`	GET	SYSTEM_ERROR	Always 500. Anomaly injection target.
`/api/random`	GET	SYSTEM_ERROR / DATABASE_ERROR	~25% random failure rate.
`/api/db`	GET	null	Real SQLite query on seeded users table.
`/api/db?fail=1`	GET	DATABASE_ERROR	Queries nonexistent table.
`/api/validate`	POST	VALIDATION_ERROR	Joi: email (required, valid format) + age (int, 18-60).

---

7. Conclusions

All deliverable requirements have been satisfied. The stack demonstrates structured logging with a stable schema, RED metric instrumentation with correct histogram bucket design, centralised error categorisation including the non-trivial TIMEOUT_ERROR-on-200 case, and a controlled anomaly experiment with machine-readable ground truth.

The most operationally significant design decision is the decoupling of error_category from HTTP status code. A system that alerts only on 4xx/5xx responses is blind to latency degradation — the failure mode that most commonly precedes full outages. By classifying slow-but-successful responses as TIMEOUT_ERROR in both the log and metric layers, the stack enables SLO-based alerting on latency budgets independently of error budgets.

The anomaly window marker pattern — a Prometheus gauge toggled by the load generator — is directly applicable to production canary deployments, feature flag rollouts, and chaos engineering experiments where precise, machine-readable window boundaries are required for automated regression detection.